

IT Project Guidance

Integrations – Overview of Common Approaches

Version: 1.3

Purpose

This document provides design guidance for selecting integration approaches between digital services. It explains a wide range of commonly encountered approaches, including constrained and legacy techniques, so that design decisions can be made with a clear understanding of their purpose, trade-offs, and long-term consequences. The document prefers purpose-designed integration APIs where feasible, while recognising that budget, product maturity, contractual limits, or legacy system constraints may require less desirable alternatives.

Synopsis

Integration design is often discussed as if it were mainly a technology choice. It is not. The first question is what purpose the interface is intended to serve. An interface designed for system-to-system integration is not the same as one designed to support a user interface, reporting, or internal processing. Confusing these purposes leads to brittle designs, hidden coupling, and poor long-term maintainability.

This document classifies common integration approaches across both modern and constrained environments. It distinguishes interface purpose from interface technology, and it distinguishes internal logical data structures from externally consumable conceptual views. It shows why purpose-designed integration APIs are generally the preferred strategic option, while also explaining where UI-based, file-based, agent-based, or direct-access approaches may still be encountered. The document concludes with anti-patterns to avoid and recommendations for selecting and governing integration approaches in a disciplined manner.

Contents

Purpose	1
Synopsis	1
Contents	2
Background	5
Disambiguation of Terms	5
Purpose	6
Context	6
Modelling Maturity: Internal Models and External Contracts	7
Topologies for Integration and Interoperability	8
Point-to-Point Topologies	9
Advantages	10
Considerations	10
Disadvantages	10
Gateway-Mediated Topologies	11
Advantages	11
Considerations	11
Disadvantages	11
Integration Platform or Suite-Mediated Topologies	12
Advantages	12
Considerations	12
Disadvantages	13
Data Warehouse-Based Topologies	13
Advantages	13
Considerations	13
Disadvantages	14
Datastore-Based Topologies	14
Advantages	14
Considerations	14
Disadvantages	15
Operational Data Hub Topologies	15
Advantages	15
Considerations	16
Disadvantages	16
Distributed Operational Data Hubs	16
Advantages	17
Considerations	17
Disadvantages	17
Recommended Direction of Travel for Topologies	18
Common Interface and Exchange Methodologies	18
User-Mediated Interface Methodologies	18
Hypertext Links	18
Advantages	19
Considerations	19
Disadvantages	20
Purpose-Built Interface Methodologies	20

Webhooks.....	21
Advantages.....	21
Considerations.....	21
Disadvantages.....	22
Integration APIs.....	22
Advantages.....	23
Considerations.....	23
Disadvantages.....	25
UI APIs.....	25
Advantages.....	26
Considerations.....	26
Disadvantages.....	26
Constrained and Storage-Led Exchange Methodologies.....	27
Physical Transfers.....	27
Advantages.....	27
Considerations.....	27
Disadvantages.....	28
SFTP.....	28
Advantages.....	28
Considerations.....	28
Disadvantages.....	29
Shared Cloud Storage Exchange.....	29
Advantages.....	29
Considerations.....	29
Disadvantages.....	29
Direct Datastore based ETL.....	29
Advantages.....	30
Considerations.....	30
Disadvantages.....	30
Integration Agents.....	31
Advantages.....	32
Considerations.....	32
Disadvantages.....	32
Integration Agents through Tunnel Agents.....	32
Advantages.....	33
Considerations.....	33
Disadvantages.....	33
Anti-patterns to Avoid.....	34
Recommendations for Maturing Integration and Interoperability Capability.....	34
Conclusion.....	35
Appendices.....	36
Appendix A - Document Information.....	36
Authors & Contributors.....	36
Versions.....	36
Images.....	36
Tables.....	37
References.....	37

Review Distribution37

Audience.....37

Structure37

Diagrams37

Terms.....37

Appendix B - Information Flow Modelled as Reticulated and Storm Water39

Appendix C - Extreme Case of Boundary, Schema, and Control Muddling.....40

Background

Integration is a necessary part of digital service delivery, but projects rarely begin from an ideal architectural position. Many existing systems, especially older or packaged products, were not designed to provide purpose-built interoperability interfaces. Their capabilities may be exposed only through user interfaces, file exports, database structures, or other arrangements that reflect the era and assumptions under which they were built.

Even where stronger methodologies are technically available, they are not always usable well in practice. Budget limits, vendor constraints, contractual boundaries, procurement timing, and the operational risk of changing live services all narrow what can be done in the near term. More importantly, the organisation may not yet have the maturity, governance, ownership clarity, or target topology needed to support those methodologies properly. A method can be available without the estate being ready for it.

That is why clarity matters. Teams under delivery pressure too easily mistake what is available for what is appropriate. Enterprise and solution architecture can make the same mistake when they focus on the presence of a technology or product without first establishing whether the surrounding topology, stewardship, lifecycle management, and information boundaries are mature enough to sustain it. The fact that something can be connected does not mean it provides a sound interface, a suitable topology, or a sensible direction of travel.

This guidance is therefore deliberately pragmatic, but not neutral. It covers both the topologies by which integration and interoperability are arranged, and the methodologies used within them, because real projects must often choose among imperfect options. Some of those options are acceptable stepping stones. Others are merely expedient compromises. They are not all equal, and they should not be treated as such.

The purpose of this document is to help delivery teams, enterprise architects, solution architects, decision-makers and governance recognise that distinction, choose more deliberately, and move over time toward brokered, governed, and more valuable use of information.

Disambiguation of Terms

Integration is the deliberate act of connecting one system to another to depend on and consume the source system's capabilities, data, or events.

Interoperability is the deliberate provision of stable, understandable, and governable access to an **interface** to a conceptual model of the system's resources to approved external consumers.

Interface is the boundary through which that access or dependence is expressed.

The distinction matters because it separates the act of dependency from the capability being deliberately offered, at a specific point.

Purpose

The first design question is not whether an API, file transfer, or agent can be used. It is what the interface is *for*.

In broad terms, interfaces usually exist for one of three purposes. Some support dependable system-to-system exchange. Some support a live user experience through views, controls, and session-bound interactions. Others support reporting, extraction, reconciliation, or downstream analysis.

These purposes create different design pressures. Integration interfaces need stability, governance, and controlled evolution. UI-facing interfaces are shaped by responsiveness, session state, and immediate user context. Reporting and extract interfaces are shaped by completeness, freshness, and query needs. These categories can overlap at the edges, but they should not be casually merged. An interface that is fit for one purpose is often a poor choice for another, and treating them as interchangeable is a common source of weak design.

Context

Context is not background scenery. It is often the real reason weak integration choices are made. Methodologies are not chosen in the abstract. They are chosen inside an estate that already has a level of maturity, a set of available topologies, and a degree of clarity or confusion about ownership, governance, and information boundaries. If those conditions are weak, then even technically available methods are unlikely to be used well.

Immaturity rarely appears in only one place. It tends to show up together in weak modelling discipline, weak topology choices, and weak methodology choices. Where ownership is unclear, information boundaries are poorly brokered, and governance is immature, teams are more likely to expose internal shapes directly, rely on brittle point-to-point arrangements, and choose whatever method seems immediately possible rather than what is strategically sound. Poor modelling maturity leaves systems unable to distinguish properly between internal logical structures and externally consumable conceptual views. Poor topology maturity leaves stronger patterns such as gateways or operational data hubs absent, so direct ETL, point-to-point, or constrained datastore access become the practical default. Poor methodology choices then follow from those earlier weaknesses rather than from the real purpose of the interface.

Immature estates also tend to mistake the wrong architectural component for an integration point. A common example is treating a data warehouse as though it were an operational interoperability boundary when it is really shaped for analysis and

downstream accumulation. That kind of confusion reinforces weak design by making consumers depend on structures that were never intended to serve as external contracts.

For that reason, the document next addresses the three connected maturity concerns in order:

- It first addresses modelling, because poor external contracts usually begin with poor distinction between internal and external information shape.
- It then addresses topologies, because methodologies are constrained by the broader arrangements available across the estate.
- Only after that does it address methodologies, because methodology choice is the consequence of context and maturity, not a starting point in its own right.

Modelling Maturity: Internal Models and External Contracts

This paper is not primarily about data modelling – while foundational to this discussion, that subject is addressed more fully in a separate guidance on the matter. Even so, one point must be understood here without fail.

Good system design distinguishes between conceptual, logical, and physical representations. This tiered distinction is long established in information architecture, including the ANSI/SPARC three-schema model, and remains important because each layer serves a different purpose.

In plain terms, the conceptual layer describes the enduring business ideas the organisation cares about in a form that is recognisable to the business stakeholders who fund and use the service. It is the watch face: the part people can read and make sense of without needing to understand the mechanism beneath. The logical layer then describes how the system chooses to organise those ideas for behaviour, roles, rules, and processing. It is the watch gearwork: the internal mechanism that makes the visible result possible. The physical layer finally reflects how that logical design is persisted in technology, so that information can be stored, retained, and carried across time between logical operations.

For example, a conceptual view that presents a student enrolled at a school is immediately recognisable and therefore attractive to users and stakeholders. However, recognisability is not enough. That simple view does not cope well with the fuller and changing reality in which the same person may later also be a parent, a teacher, part of the workforce, or connected to more than one group or institution over time.

The logical design must then carry the heavier burden of representing participation, role, relationship, and change over time. It may therefore introduce structures such as person,

membership, role, learner profile, school profile, or other role-specific records so that the same individual can move across multiple groups and roles without forcing those changing circumstances into the core concept itself. That added logical structure is not unnecessary complexity. It is what prevents today's temporary concerns from being baked into tomorrow's core concepts and external contracts. In disciplined design, the conceptual view stays stable for consumers while the logical design stays free to evolve.

In practice, many modern systems no longer maintain those distinctions well.

Development tooling has made storage schema creation so easy that it can give the appearance of mastery where the underlying modelling discipline is weak. Low-code and no-code platforms often go further again by serialising business-shaped objects directly back and forth between storage and user interface concerns, with little or no brokering across layers. The result is that internal models, whatever shape they happen to take, are too easily mistaken for suitable external contracts. That should be resisted. By both training and governance. Objects exposed for integration should be brokered from the internal model into an intentionally designed external contract, not passed through directly from internal structures whether those structures are conceptual-looking, logical, or close to physical storage. The reason is not only current coupling. It is the loss of evolvability. Direct exposure encodes today's business concerns, internal assumptions, and present organisational structure into the interface itself. Those concerns rarely remain the same over time, even in slow-moving organisations. A brokered and more deliberately abstracted external contract reduces that risk by separating what consumers need to use from how the source system currently happens to organise itself and persist information.

Exposing internal logic in any form hardwires one system's present concerns into another system's future. The result is tighter coupling, slower change, higher risk, and higher cost. But the larger cost is organisational. Once a system models around role-specific silos such as student or teacher, it stops reflecting reality and starts dictating structure. Separate services, separate teams, and separate ownership lines then grow around what was only ever a temporary modelling convenience. Hesitancy to change follows, because change now threatens not only software, but the boundaries of the organisation itself. What should have remained free to evolve becomes trapped in dependencies, silos, and institutional inertia.

Topologies for Integration and Interoperability

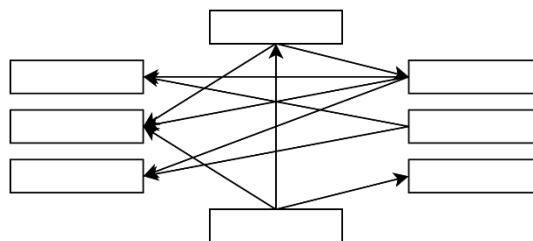
Before considering specific interface and exchange methodologies, it is necessary to be blunt about topology. For years, integration thinking was organised around the system of record and the record of truth. Those ideas remain important, especially for authority, provenance, and legal source - but they proved too narrow as organising concepts for broader interoperability and operational use.

The design instinct that ensued became to only retrieve information directly from the authoritative source rather than to broker information for use. That led too many organisations toward direct source-system dependency, whether by ETL or API. That approach may be workable in limited cases, but it is not a sound long-term strategy for a growing estate. It encourages point-to-point patterns, embeds source-system primacy into downstream design, and makes each new consumer another dependency to preserve.

Gateway-centric thinking later improved control over exposure, documentation, throttling, and policy -- but was often mistaken for a sufficient architectural answer. It was not. A gateway can improve access management, but it does not solve poor modelling and contract design, weak information boundaries, or confusion about interface purpose. It also adds distance into the path. Every extra hop, policy check, transformation, and redirection introduces more delay, more places for failure, and more moving parts to understand when something goes wrong. For a reader unfamiliar with latency, the practical picture is simple: each extra layer is another queue, another checkpoint, or another doorway that information must pass through before it becomes useful. That may be acceptable when it brings real value, but not when the layer exists only because the estate has confused control with architecture. Nor does a single-estate gateway mindset adequately reflect a modern environment in which multiple clouds, external SaaS providers, and distributed ownership are unavoidable.

Topology therefore matters because it determines whether the estate is organised around direct dependence on source systems, or around governed and brokered use of information across a changing and distributed environment.

Point-to-Point Topologies



A point-to-point topology exists where one consumer connects directly to one provider for a specific need. It is often the first topology to appear in an estate because it is easy to justify locally and can be delivered without broader architectural maturity. That same immediacy is also its weakness, because it solves one connection at a time while quietly increasing the number of dependencies that must later be understood, governed, and preserved.

Figure 1: Point to Point Topologies

Advantages

Point-to-point arrangements can be effective for isolated temporary needs because they keep the path short, the responsibility visible, and the delivery overhead relatively low at the outset.

Considerations

Point-to-point topology is often the design of choice in immature estates, not because it is strategically strong, but because stronger alternatives do not yet exist. In practice it is commonly realised through ETL, file movement, or other direct extraction methods because the source systems do not provide purpose-designed integration interfaces. That absence is usually not an isolated technical gap. It reflects the same immaturity in data and integration architecture that makes point-to-point seem necessary in the first place. In theory, repeated demand for such integrations should create pressure for providers to offer better interoperability. In practice, that pressure often fails to convert into provider-side improvement because the funding usually sits with the consumer who needs the integration, not with the provider who would need to redesign the interface for broader reuse.

Disadvantages

Point-to-point topology scales badly in both technical and human terms. As the number of consumers grows, it duplicates effort, embeds source-system importance into downstream design, and creates brittle estates in which every new consumer becomes another dependency to preserve. The difficulty is not only technical. Integration pain between different workgroups creates friction in funding, agreement, and delivery, which in turn encourages teams to avoid building proper interoperability and instead hitch-hike on integrations or intermediate data sources someone else has already paid for. That may look pragmatic, but it makes orchestration subtle and difficult because downstream processes now depend on a mixture of partially current copied data and some remaining direct point-to-point interactions. Once that happens, two further failure modes often appear. Either governance reacts by forcing a return to direct source-system dependency in the name of purity, or teams conclude that if copied data is already accessible they may as well go to the largest copied store available, such as a data warehouse. Both responses miss the real problem and reinforce weak architecture rather than maturing it.

Gateway-Mediated Topologies

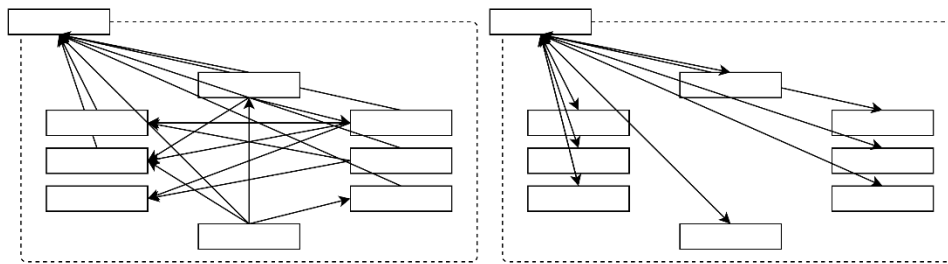


Figure 2: 2 Phase addition of Gateway, then replacement by Gateway

A gateway-mediated topology places a managed entry point between consumers and providers. It typically appears when an organisation wants more consistent exposure, routing, policy enforcement, or documentation for externally consumable services. That can be useful, but it also creates two common confusions. The first is to mistake central control of access for improvement of the underlying contracts and information boundaries. The second is to mistake one boundary control point for the architectural arrangement as a whole, as though controlling the edge were the same thing as organising the estate.

Advantages

A gateway-mediated topology can improve consistency of routing, access control, throttling, and external exposure, especially where services are intended for broader or longer-term use.

Considerations

A gateway should be treated as an access and policy layer, not as a substitute for good contract design. It can improve consistency of exposure and help separate consumers from internal endpoint change, but its role, ownership, and support expectations should remain clear. Without that discipline, a gateway becomes another place to hide poor modelling, unclear responsibility, and unresolved interface purpose behind a central technical façade.

Disadvantages

A gateway does not by itself create a good interface, a good topology, or a mature estate. If the underlying contract is poorly shaped, tightly coupled, or confused in purpose, placing it behind a gateway merely centralises access to a weak design while adding cost, delay, operational dependency, and one more place where failure or misunderstanding can occur. It can make an estate look more orderly than it really is, while leaving the underlying architectural weakness untouched.

Integration Platform or Suite-Mediated Topologies

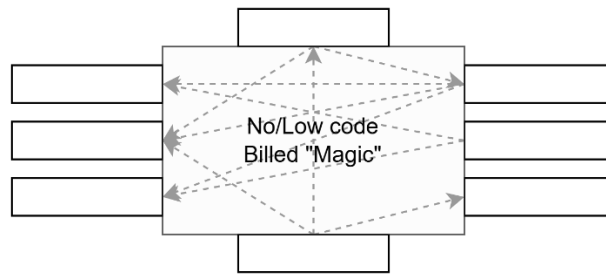


Figure 3: Integration Platform managed Topologies

An integration platform or suite-mediated topology introduces a managed intermediary responsible for routing, transformation, and orchestration across many connections. It often appears in estates where technologies span decades, protocols vary, and the current organisation no longer has enough in-house capability to bridge that complexity directly. In that sense it can be useful. But the harsh reality is that, in most estates, it does not materially improve the topology. In most cases it remains managed point-to-point: the same dependency pattern, only disguised behind a product and an operating model.

Advantages

Integration platforms and suites can be useful where there are many endpoints, varied protocols, or significant orchestration and operational integration demands. They can improve operational visibility and reduce repeated plumbing effort.

Considerations

These platforms can make some legacy integration work easier to deliver, including where low-code or no-code tooling is used to compensate for loss of current integration capability. That convenience should not be confused with maturity. The platform still does not decide business meaning, fix weak information boundaries, or create a stronger organising topology for the estate. Funding spent on managed point-to-point orchestration is funding not spent on improving the topology itself, or even on the more mundane but necessary task of keeping legacy protocols and source dependencies current and supportable.

A further risk arises where the managed intermediary is not merely tooling, but a vendor-controlled boundary on which the organisation or sector begins to depend. Delivered as SaaS or not, that does not become strategically sound unless the organisation retains meaningful control over the shared schema, practical rights to use the data, and a credible ability to move the boundary later. Without that, the arrangement can leave it carrying cost, dependency, and reputational risk without the corresponding strategic benefit.

Disadvantages

If allowed to drive design, integration suites do not remove the underlying weakness. They hide it. The estate still depends on a web of pairwise connections, only now those connections are mediated, licensed, and operationally wrapped by a platform. That can make the architecture look more controlled than it really is while detouring funds, attention, and urgency away from topology improvement. Worse, once enough managed point-to-point logic has accumulated in the platform, it becomes a brake on change. Upgrading a source system, retiring a legacy protocol, or improving a dependency boundary becomes harder because the platform itself has become another estate that must be preserved. What began as a way to cope with disorganisation can therefore become an expensive mechanism for preserving it.

Data Warehouse-Based Topologies

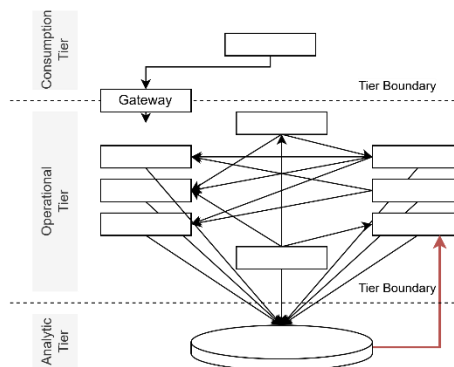


Figure 4: Data Warehouse as Source

A data warehouse-based topology uses a copied and centrally managed analytical store as though it were an operational integration point. It often appears because the data is already present, queryable, and easier to obtain from one place than to negotiate from several live systems. That convenience is real, but it risks confusing downstream analytical accumulation with a governed operational boundary designed for direct use.

Advantages

Data warehouse-based integration can appear attractive because copied data is already available, queryable, centrally managed, and often easier for upstream teams to obtain access centrally once, than negotiate access to multiple live source systems.

Considerations

A data warehouse is usually shaped for analysis, reporting, and historical aggregation rather than governed operational exchange. If it is used as an integration point, that should be recognised as a constrained pattern rather than confused with a proper interoperability boundary.

Disadvantages

Treating a data warehouse as an operational integration point is usually flawed. It encourages consumers to depend on copied and transformed data that was not designed as a live external contract, separates them from the operational meaning and timing of source systems, and can conceal unresolved orchestration problems behind the convenience of central access. It is a storage-led answer to a boundary and contract problem. A data warehouse is valuable for downstream analytical use, but that does not make it a sound operational integration boundary. It is closer to runoff collected for later analysis than to a governed supply intended for direct operational consumption.

Datastore-Based Topologies

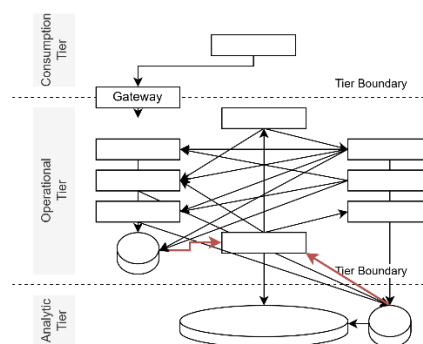


Figure 5: Data Store based Topologies

A datastore-based topology gives consumers direct or coarse controlled access to the live operational storage of a source system. It usually appears where no better external interface exists and where near-term delivery pressure makes direct store access seem like the only practical option. What is being consumed in this topology is not a deliberate external contract, but the internal persistence structures and timing characteristics of the source system itself.

Advantages

Datastore-based arrangements can provide a practical fallback where no suitable interoperability interface exists and where direct access to an operational store is the only feasible near-term option.

Considerations

Where this topology is used, it should be treated as constrained access to the live or operational internals of a source system, with explicit controls for security, auditability, schema change, and operational impact. It should not be mistaken for a well-formed external contract.

In some estates this is simply an attempted operational data hub built with only database tools and data warehouse capability. When storage technology and warehouse resources are the only tools on hand, the result is usually a store-centred imitation of a hub: data collected and exposed from one place, but without the governed logic, boundary

discipline, or external contract of versioned integration API endpoints that make a true operational data hub materially stronger.

Disadvantages

Consumers become dependent on operational storage structures, local conventions, live source-system behaviour, and the timing characteristics of the source environment itself. This creates high coupling and makes future improvement of the source system difficult, risky, and expensive.

Operational Data Hub Topologies

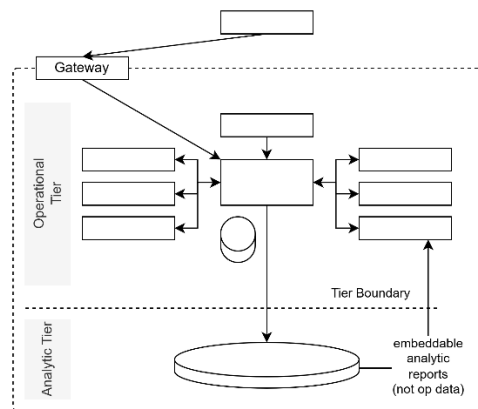


Figure 6: Operational Data Hubs

An operational data hub topology introduces a separate operational boundary designed to broker information from one or more source systems for downstream use. It usually appears where direct provider interfaces are absent, insufficient, or too tightly coupled to serve as a durable organising point for the estate. Its value lies in shifting the design focus away from source-system primacy and toward governed, reusable information shaped around how it needs to be used.

Advantages

An operational data hub introduces a deliberate boundary for governed external consumption. Unlike an integration platform, it is not only coordination tooling. Unlike a datastore or data warehouse, it is not only storage. Its major advantage over point-to-point is persistence with purpose. By persisting information from multiple source systems, it can solve a merge once and then share the resulting, more valuable downstream objects with multiple consuming services. That is materially stronger than separate consumers each going back to the same sources and each performing their own merge for their own immediate needs. In point-to-point estates, those separate merges inevitably diverge. The same facts are combined in slightly different ways, and that variance later reappears as reconciliation pain downstream, often in analytical stores. An operational data hub reduces that repeated effort by solving the merge once in a governed operational boundary and then reusing it. In doing so it also shifts design attention away from the source system and toward the information itself, which is the real object of use

because information exists to support decisions, not merely to preserve the primacy of the system that first collected it.

Considerations

An operational data hub still requires careful information design, governance, and investment. It does not remove the need for source-system stewardship or clear decisions about refresh timing, ownership, and quality. In practice, this also means resisting the temptation to make one hub do everything. Multiple smaller hubs aligned to divisions, domains, or similarly sized areas of responsibility are often more useful because they can focus on the needs of that area without forcing every change through one central structure. They also reduce cross-hub communication to what is actually necessary, separate risk, and make the estate easier to change when organisational structures inevitably evolve. If a division is restructured, separated, or divested, it is far easier for it to leave with its own hub than to extract itself from one oversized and entangled central platform.

Disadvantages

This topology requires more up-front discipline and usually more time and funding than point-to-point arrangements. The gap is often not desirability but timing and available resources.

Distributed Operational Data Hubs

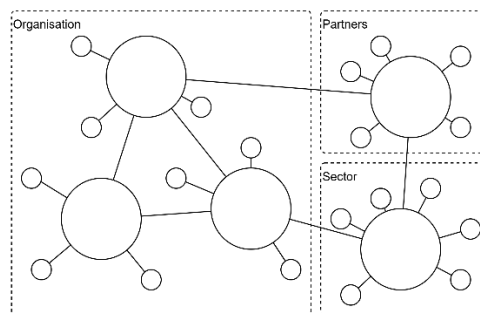


Figure 7: Distributed Operational Data Hubs

A strong operational data hub pattern is usually distributed rather than universal. Operational data hubs work best when aligned to natural organisational or jurisdictional boundaries and then connected through disciplined exchange. That applies internally as well as externally. Within one enterprise, separate hubs may be appropriate for divisions, departments, or similarly sized areas of responsibility, especially where restructuring, separation, or divestment may later occur. Across harder boundaries such as sectors or countries, the case is stronger still. Different legal regimes, sovereignty requirements, and obligations can materially change what must be stored, exposed, retained, or governed. The better design move is usually to keep separate hubs and optimise the efficiency, timing, and quality of exchange between them.

Advantages

Distributed operational data hubs preserve the strengths of an ODH while avoiding the fragility of one universal hub. They allow local needs to be met more directly, separate risk, keep change more governable, and make it easier for one part of the estate to evolve without forcing every other part to move at the same time. They also provide a clearer basis for disciplined exchange across internal and external boundaries because each hub remains aligned to a real domain of responsibility rather than a forced universal abstraction.

Considerations

This issue is especially important where the apparent domain boundary is temporary, conditional, or only partly aligned. A central internal hub should not simply be shared across a wider sector as though sector partners were departments of the same enterprise. They are not. At best they are separate organisations whose obligations, strategies, budgets, and operating realities may align for a time, but are never guaranteed to remain the same. The same logic applies internally. One division, department, or business unit should not be forced into a single hub merely because the organisation chart currently places them together. If that structure later changes, the cost of disentangling one oversized hub can be severe. The same caution applies where one part of a sector is mistaken for the whole, or where a hub designed around one internal domain is assumed to suit tertiary, vocational, or other adjacent domains simply because they also deal in related information. Across countries, the case is stronger again. Different sovereignty requirements and legal regimes, including constraints that may arise through local law or extraterritorial reach such as the Cloud Act, can materially change what must be stored, exposed, retained, or governed.

The useful analogy is roundabouts. A shopping centre may have its own roundabout, just as the city has its own, and a neighbouring private precinct may have another. Trying to make all three the same, under one ownership and one budget, is usually not realistic. The better effort is spent making movement between them efficient and well-controlled.

A further distinction also matters here. There is a material difference between using SaaS technology as a tool and allowing an external SaaS provider to become the practical owner of schema, engagement, and boundary operation without a credible exit path. That is like handing one of the roundabouts to an external operator without retaining control over whether it continues to meet the road-safety standards required by the city. The first may be a delivery choice. The second is a loss of architectural control.

Disadvantages

Trying to force one operational data hub across divergent divisions, sectors, jurisdictions, strategies, or legal regimes usually produces the wrong compromise. The hub becomes slower to change, harder to govern, and more politically fragile because each participating domain drags different requirements into the same shared structure. Instead

of improving interoperability, the oversized hub becomes a contested bottleneck and a new form of monolith. That effort is better spent on strong local hubs and disciplined exchange between them.

Recommended Direction of Travel for Topologies

The direction of travel should be stated plainly. Point-to-point topology should be avoided as a strategic pattern, even when it is partially obscured by gateways, integration platforms, or other managed intermediaries. Those additions may improve control or operability, but they do not change the underlying dependency model. The preferred direction is toward brokered and governed topologies that reduce direct dependence on source systems. Where an organisation can fund and build purpose-designed integration interfaces, that should be encouraged. More broadly, the topology to be preferred is not one universal hub, but smaller operational data hubs aligned to natural divisions, domains, or similarly sized areas of responsibility. That is usually more pragmatic, allows focused needs to be met without over-investing in one technology stack, and reduces the risk of creating migrations so large and monolithic that they are deferred indefinitely. Change is inevitable. The better design move is to prepare for it by keeping hubs smaller, more governable, and better connected through disciplined exchange between them.

Common Interface and Exchange Methodologies

The following sections describe common interface and exchange methodologies used when systems need to connect, exchange information, or provide external access. These methodologies can be employed within different topologies and for different purposes, so they should not be evaluated in isolation from the broader arrangement in which they are used. Some support stronger interoperability or structured external access. Others are constrained or fallback methodologies used where better options are absent. Each is considered in terms of advantages, considerations, and disadvantages so that design choices can be made consciously rather than by accident.

User-Mediated Interface Methodologies

These methodologies depend on a user's active participation in navigating between systems. They are important because they often shape first contact, discovery, and continuity of service, but they do not by themselves provide dependable system-to-system exchange. Their value is therefore mainly in mediated access and user journey support rather than in governed interoperability.

Hypertext Links

The simplest integrations between web services are user-activated hypertext links from one view within the user interface of one service to another view within the same or

another service. They are humble in mechanism, but important in practice because they shape how users discover, enter, and move between services.

Hypertext links should not be dismissed merely because they are simple. They are foundational to how the web works and are unlikely to disappear. In many services they also form the user's first interaction with a system, often by taking a person from a search result or corporate information page into the service itself. That makes them strategically important to discovery, navigation, and service take-up. A poor linking experience at that first point of contact can reduce confidence and discourage further use, even where the underlying service is sound.

Examples include incoming linking a project site from a corporate intranet, extranet, or product brochure site. Outgoing links may connect users from within an application or project site to external resources such as the product's help website or a feedback service, supporting user self-service or collection of user input.

Hypertext links are also used to connect from an intranet or extranet corporate site to a project's dedicated static information site.

From there, links are used to connect to one of the following targets:

- A public view of the solution's system
- A protected view of the solution's system, which redirects it to its login form,
- An IdP service separate from the target service, passing information as a query param to indicate where to return to¹.

Advantages

Hyperlinks are simple to implement and require little technical machinery because the interaction is initiated by the user rather than by timers, schedulers, or background processes. Their value, however, is not merely technical simplicity. They are foundational to web navigation, often provide the user's first transition into a service, and can materially influence discovery, confidence, and service take-up.

Considerations

Hyperlinks are capable of passing limited contextual information—such as the identifier of the current view or control within an application. For instance, when a user selects a help link, information about the specific page being viewed can be sent to the help service, enabling it to display targeted support content.

Quality assurance through automated user interface testing tools (e.g., Tosca, Selenium) is particularly challenging; tests can be brittle, difficult to maintain, and not easily

¹ To avoid design brittleness and maintenance requirements it is recommended that links avoid passing information.

integrated into automation pipelines. As a best practice, it is recommended to avoid UI automated testing when possible and instead focus on testing the application logic of the APIs used by the view. This approach ensures the functionality is tested directly, rather than UI rendering or pre-validation.

Hyperlinks are also often used to begin an authentication flow. A link may direct the user to the target service itself, to an identity provider, or to an intermediary page that starts sign-in and then returns the user to the intended destination. In practice this is commonly achieved by redirect-based federation patterns, such as OpenID Connect or Security Assertion Markup Language, where the original destination or application state is preserved and returned after successful authentication. This can provide a smooth transition for the current user, but it also means link design must account for redirect handling, state validation, and safe return paths rather than assuming navigation alone is enough.

Sensitive information should not be placed directly into URLs unless there is a very deliberate and well-controlled reason to do so. URLs can be stored in browser history, server logs, analytics tooling, bookmarks, referrer headers, and copied messages. Where context must be preserved across a link or sign-in flow, it is better to pass opaque references or controlled state values than to expose business data or credentials directly.

As with all channels enabling integration and interoperability, hyperlinks must be secured. It is essential to use secure HTTP (HTTPS) to protect the channel.

Disadvantages

Hyperlink-based integrations depend on user action, so they do not provide reliable or timely execution of an integration process. If the user does not follow the link, no exchange occurs.

Any information carried by the link is typically limited and pre-encoded into the client interface. This makes the interaction shallow, static, and poorly suited to controlled data exchange, because the structure and meaning of what is passed are usually constrained by the link design rather than by a governed integration contract.

Hyperlinks are also brittle over time. URLs, routes, redirect patterns, and authentication entry points change as websites and services evolve. Without deliberate governance, redirection management, and routine validation, links can silently degrade or fail, damaging user trust and weakening service access at the very point where many users first encounter the system.

Purpose-Built Interface Methodologies

These methodologies provide explicit interfaces intended for external consumption or structured exchange. They are generally stronger than user-mediated or storage-led approaches because they can offer a clearer contract, better security boundaries, and

more controlled evolution. Even so, they still need to be matched to the real purpose of the interface rather than assumed to be appropriate merely because they are technically APIs.

Webhooks

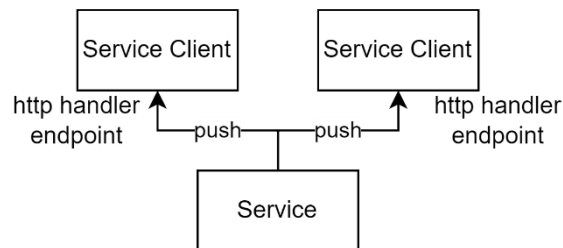


Figure 8: Web hooks based integration

With server-offered APIs, such as REST, service clients are required to continuously poll the server to check for the latest changes. This polling process can significantly increase network traffic, reducing available I/O threads and bandwidth for processing user requests, or alternatively, introduce delays when attempting to conserve bandwidth. The impact of this approach is magnified when there are large numbers of service clients, each independently making requests to the server.

Webhooks provide a solution by shifting the responsibility to the server, which actively notifies registered clients of changes by sending requests to their webhook endpoints. This eliminates the need for client-based polling. Webhooks function as one-way, publicly accessible integrations that do not require the prior establishment of a user session.

Additionally, webhook calls are asynchronous and allow the server to send notifications to clients without waiting for a response, making the integration process more efficient and streamlined.

Note:

The approach is classifiable as a PUSH pattern, as opposed to a PULL pattern.

Advantages

Webhooks² remove the need for cyclical polling, decreasing unnecessary traffic and IO consumption, as well as the delay before appropriate clients are notified of changes on the server.

Considerations

Idempotency: Webhooks for the most part should be POST HTTP verb based as they are providing information, not retrieving information, which would require a GET. Less

² Sometimes colloquially referred to as 'reverse APIs'.

common, and more of an anti-pattern, Webhook handlers can be triggered from UI HTML GET based Links.

Response: Webhooks may have response payloads, but it is less common than making the webhook appropriate for a non-acknowledged “fire and forget” call.

Confidentiality: As with any connection between devices, especially across multiple networks, it should be secured using HTTP/S over TCP/IP domain-based TLS certificate.

Integrity: The validity of payloads can be verified if a secret has been provided to the service so that it can develop a cryptographic hash signature of the payload.

Dependencies: For a web-based service to hook a webhook endpoint on a client system on a private network, a reverse proxy is used.

Portability: If used by services external to the providing organisation, consider making the endpoints be redirected through a Gateway.

Response Codes: consider sending and being prepared to process HTML response codes.

Disadvantages

Few systems are developed with integration APIs, even less with webhook endpoints.

There is no industry standard or common pattern for the development of either webhooks or their payloads, so each can have their own quirks and associated development and testing costs.

Without acknowledgement messages, the server may believe that clients have been notified.

Integration APIs

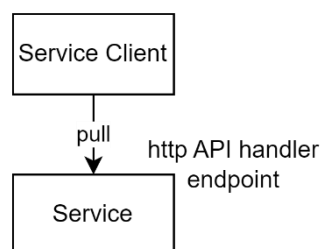


Figure 9: API integration approach

Application Programming Interfaces (APIs) provide an explicit boundary through which one system can request information from, or submit change to, another. For interoperability design, their value lies less in the fact that they are technically APIs than in the fact that they can provide a governed contract for external use. Where HTTP resource-oriented APIs are used, the dominant architectural style is REST (Representational State Transfer), as originally defined by Roy Fielding in his dissertation on network-based software architectures.

API protocols typically follow one of two primary approaches: invoking Procedures or managing Resources.

Remote Procedure Call (RPC) based APIs enable systems to execute a wide range of commands with varying signatures. In contrast, Resource-based APIs focus on enabling Create, Read, Update, and Delete (CRUD) operations on defined system resources.

Generally, RPC-based protocols like SOAP offer greater complexity and flexibility, making them valuable for interactions within the same environment. On the other hand, the straightforward nature of Representational State Transfer (REST) has made it especially useful as the standard for APIs exposed to external third-party service consumers.

REST constrains remote clients to use one of four basic HTTP verbs (operations) on a specified resource. For example, a client retrieves information by sending a GET request to a resource, or submits or modifies data by using POST, PUT, or DELETE verbs as necessary.

Given the scope of this document in addressing third-party integrations, and the move away from use of SOAP for current development, the focus will remain solely on REST APIs.

Note:

The approach follows a PULL as opposed to a PUSH pattern, providing more specific control to the recipient as opposed to the provider.

Advantages

Unlike Webhooks, services that offer REST endpoints do not rely on the need to access the remote client system directly.

Because REST APIs are based on HTTP/S, requests can be routed through a Web Application Firewall (WAF), providing an additional layer of security.

The widespread adoption of REST-based API architectures means that a wealth of market resources, tools, development patterns, and standards exist to simplify development and testing.

Beyond OData, there is no universally accepted approach for implementing qualifiers to verb-based actions; common functionalities such as filtering, projecting, sorting, and paging are handled according to individual choices.

Considerations

Channel Security: APIs must operate over HTTP or HTTPS to ensure secure, reliable communication between distributed clients—even across public internet infrastructure. Adhering to these standards promotes interoperability and protects data integrity throughout transmission.

Authentication and authorisation: Implementing robust authentication and authorization solutions—such as OAuth or JWT—is vital for verifying client identities and managing secure access to API resources. Authentication may be required at both the gateway and the service.

Gateways: long standing public APIs should be available through redirection provided by an API gateway. They enable rerouting of API traffic, thereby supporting maintenance and evolution of services without requiring clients to modify their integrations, a crucial outcome when APIs are available outside of the organisation. However, in enterprise settings, where systems are rarely replaced, their materialised benefits may remain more hypothetical than materialised. They also introduce latency, infrastructure, bandwidth consumption and licensing costs.

Rate limiting: To prevent abuse and ensure fair distribution of resources, APIs should enforce limits on the frequency of client requests.

Lifespan Management: APIs should clearly communicate their intended support lifecycle, signalling deprecation and end-of-life milestones well in advance. This transparency enables consumers to plan migrations and adaptations effectively.

Versioning: Since consumers may continue to use older API versions despite repeated prompts to upgrade, establishing clear versioning practices is essential. Defining the API version as a header or argument—rather than embedding it in the path—ensures consistent access to resources across all versions.

Queryability: Adopting standardized frameworks such as OData (ISO/IEC 20802-1:2016 and related OASIS standards) provides a strong foundation for designing and consuming RESTful APIs, supporting robust and flexible query capabilities.

Separate guidance is provided on modelling and external contract design, because that subject is broader than this paper alone. Even so, the expectation should be clear here. Integration APIs are expected to align to standards, support parallel versioning so consumers can be given a defined grace period to upgrade, and provide standards-based queryability rather than bespoke parameter conventions wherever practical. In practical terms, that means predictable caching behaviour, security aligned to established approaches such as OAuth 2.0 (RFC 6749, with later security best-practice guidance in RFC 9700) rather than custom schemes, and interfaces described using an open standard such as the OpenAPI Specification. Interfaces should be designed for evolvability and expanding reuse over time. Implementers should not assume that a technically working API is sufficient if it ignores those expectations.

Idempotency : Ensuring idempotency is a critical aspect of robust API design. It is recommended to use standards-based approaches to achieve idempotency, rather than relying on custom or ad hoc solutions. When RESTful principles are applied, as in the case of OData, change operations are inherently idempotent and do not interfere with the effectiveness of caching mechanisms. Conversely, in non-REST frameworks such as

GraphQL, all operations are transmitted via POST requests, which precludes standard HTTP caching. As a result, these frameworks do not natively support idempotency or cacheability, requiring more compute and storage infrastructure and costs, emphasizing the importance of adhering to established standards whenever possible.

Principled design: Once third-party services begin using an API, making subsequent changes becomes challenging. To avoid widespread disruption, it is essential to keep data schemas concise and well-segregated, ensuring that changes in one domain do not cascade into others. Entities that endure over time, such as “people,” should remain distinct from relationships (like “studentAt”) and profiles (“studentProfile”), rather than being combined into a single, mutable entity (e.g., “Student”).

Monitoring: Comprehensive monitoring and logging practices enable thorough tracking of API usage, enhance troubleshooting, and support system reliability.

Error Handling: Consistent error handling, accompanied by clear documentation and standardized error messages, streamlines the troubleshooting process for external developers and supports successful integrations.

Disadvantages

The inherent simplicity of RESTful APIs presents certain design challenges, particularly when operations on resources must themselves be treated as distinct resources—so-called “operation requests.” This requirement often leads to the creation of workarounds in API architecture.

When implementing qualifiers for verb-based actions, using established standards is generally preferable to adopting unique or novel solutions. OData, formalized as ISO/IEC 20802-1:2016, serves as a robust standard for this purpose. However, its adoption has faced competition from non-standard, open-source protocols such as GraphQL.

Service designers aiming to increase third-party engagement are frequently compelled to offer APIs compatible with both OData and GraphQL. This dual approach reflects the evolving landscape of RESTful API design, where compatibility and adherence to best practices remain central to fostering broader market adoption.

UI APIs

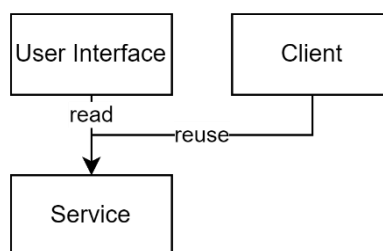


Figure 10: UI API Reuse Approach

APIs created primarily to support a user interface are not the same thing as integration APIs. They exist to serve views, controls, and user interactions within a live session, often through asynchronous requests from a browser or single-page application. Their contract, timing, tenancy, and security assumptions are therefore shaped around the active user experience, not around durable system-to-system exchange.

Unlike integration APIs, UI APIs are typically restricted to a single tenancy and a specific context, such as an individual school, and operate directly within the currently active user session.

Advantages

Separating the user interface from the underlying logic improves maintainability and allows the visible part of the service to change more quickly than the business rules beneath it. UI APIs can therefore be valuable within the boundary of the application they were designed to serve, especially where responsive user experience and front-end refresh are priorities.

Because these interfaces sit within one application boundary rather than serving broad external consumers, they are often easier to change than a true interoperability contract. That flexibility, however, is a property of their narrow purpose, not a reason to reuse them as though they were integration APIs.

Considerations

The design and operation of UI APIs should be judged against the needs of the active user experience, not against the expectations of broader interoperability. Security, tenancy, context switching, session handling, monitoring, and performance all need to be managed within that user-facing purpose. Problems arise when those same interfaces are treated as though they were suitable for durable system-to-system exchange. Once that happens, workarounds such as repeated sign-in, context cycling, and cached identifier lists become necessary, bringing brittleness, risk, and waste with them.

Disadvantages

In situations where no Interoperability API is provided, and service clients wish to use these UI APIs for integration, they must repeatedly sign in and out to access the required contexts. This approach is inefficient and time-consuming, particularly when only a single request per context is necessary.

This approach assumes that one user account is a member of all required contexts. To manage this, the service client must be provisioned in advance with a complete list of context identifiers, ensuring that a single user account has access to each. Alternatively, the client may need both a list of context identifiers and unique credentials for multiple accounts—one for each context. Both scenarios present distinct security risks.

Maintaining up-to-date lists of context identifiers and user accounts requires a dedicated refresh mechanism to keep pace with changes. However, even with regular updates, the

client must iterate through the full list, regardless of whether any changes have actually occurred on the server. Most queries, therefore, will return no new data, resulting in inefficient use of resources.

In summary, this method is not only brittle but also significantly increases the demands on both time and system resources.

Constrained and Storage-Led Exchange Methodologies

These methodologies are usually encountered where stronger interfaces are absent, where organisational maturity is low, or where external constraints prevent better options. They are not mature interoperability patterns, and they should not be treated as endorsed architecture simply because no other immediate option appears viable. At best they are constrained or transitional compromises that may need to be accepted for a time, but only with explicit acknowledgment of their cost and with a credible plan to move away from them. Technical debt that is never paid is not debt at all. It is Technical theft from the future.

When integration-specific APIs are not available – and trying to use a workaround by using UI-focused APIs proves too challenging and unreliable -- it becomes necessary to explore less optimal, non-API solutions.

Physical Transfers

An option is to manually transfer data from source system to target system via a portable datastore. Such as a thumb drive.

Advantages

There are no significant advantages to using thumb drives for manual data transfer in this context.

Considerations

When direct access to the source data is not possible—such as when the data resides within infrastructure controlled by a third party that does not permit the installation of ETL agents—manual transfer may become the only feasible solution, despite its significant drawbacks. While ETL agents are generally preferred for data integration tasks, the lack of access permissions can necessitate resorting to physical, manual methods.

Security: the drive should be encrypted.

Data Integrity: Manual processes are prone to human error, such as incorrect file selection or accidental deletion, which can compromise the accuracy and completeness of transferred data.

Version Control: There is an increased risk of version mismatches, as multiple manual copies may be created, making it difficult to ensure the target system receives the most current data set.

Compliance: Transporting data via physical media may violate organisational policies or regulatory requirements regarding data handling, storage, or cross-border movement.

Scalability: Manual transfer methods do not scale well with growing data volumes or frequency of transfers, quickly becoming unsustainable for larger operations.

Resource Allocation: Staff time and attention are diverted from higher-value activities to supervise and execute manual transfers, impacting overall productivity.

Disadvantages

Confidentiality: Manual data extraction requires direct access to the production system, which exposes sensitive information during the process of creating and transferring CSV files.

Auditability: There is no reliable way to track or audit who accessed the system or data, leading to potential gaps in accountability.

Security: Portable media such as thumb drives are at risk of being lost or misplaced during transit, jeopardizing the security of the transferred data.

Data Structure: The export process typically flattens the data structure. This can undermine relational data integrity or require a specific processing sequence to correctly import the data into the target system.

Timeliness: The time lag between data extraction and confirmation of successful receipt and usability can be significant, affecting the overall reliability of the transfer.

Effort: Manual processes demand substantial preparation, such as arranging for appropriate access rights. This makes manual transfer the least desirable option due to the high effort required.

Repeatability: Since the process is entirely manual, it is slow and labor-intensive. Participants are less inclined to repeat these steps, which negatively impacts the freshness and relevance of the data.

SFTP

An alternative to manual transfer methods is the use of a Secure File Transfer Protocol (SFTP) server, which allows secure data exchanges between systems belonging to different organisations.

Advantages

This method offers modest improvements over physical transfers by reducing reliance on portable media and enabling direct electronic transfer. SFTP can facilitate quicker data movement and enables basic access control through user credentials.

Considerations

Although SFTP provides a faster means of transfer, many of the challenges associated with manual processes persist. The process still requires both parties to set up and

maintain SFTP services, with appropriate credentials issued for data deposit and retrieval. Coordination is needed to ensure that the correct files are uploaded and downloaded, and to manage user permissions for secure data access.

Disadvantages

The disadvantages are largely the same as those found with physical transfer methods. SFTP does not address issues of auditability, repeatability, or the need for manual involvement. Data structure integrity may still be compromised if files are exported in flat formats, and delays or errors can occur if files are transferred incorrectly. Furthermore, while SFTP is more secure than using physical media, it remains dependent on the proper management of credentials and secure channels, exposing the process to risks if not handled diligently.

Shared Cloud Storage Exchange

A shared cloud storage exchange writes files into a shared object store or cloud-hosted folder from which another service later retrieves and processes them. It is often presented as a modern integration pattern because the storage surface is cloud-based, but functionally it remains a file-drop approach hosted elsewhere.

Advantages

This pattern can be quick to establish where no proper interface exists and where both parties can at least agree on a shared storage location and file format. It can also decouple runtime availability slightly, because producer and consumer do not need to be online at the exact same moment.

Considerations

Cloud hosting does not change the need for explicit control of file naming, schema versioning, retention, error handling, duplicate handling, partial-write detection, and ownership of the exchange contract. Someone must own the exchange, not merely the bucket. If not governed carefully, a shared cloud folder becomes just another staging area feeding later ETL and reconciliation effort.

Disadvantages

This should be treated as a last-resort or transitional pattern. It is still file exchange, only with a more modern hosting surface. Timing is ambiguous, completeness is uncertain, schema drift is easily hidden in files, and operational responsibility is often split awkwardly between teams and products. The apparent modernity of shared cloud storage can therefore disguise an old and weak integration pattern rather than improve it.

Direct Datastore based ETL

Direct datastore-based ETL extracts data from the operational storage of a source system and moves it into another store or processing pipeline for downstream use. It is commonly used where no integration API exists and where a consumer needs to ingest

data on a recurring basis. Although familiar and often technically straightforward, it is still a constrained methodology because it depends directly on the source system's persistence structures rather than on a governed external contract.

Advantages

Direct ETL can be effective where the data need is primarily extractive, no suitable interface exists, and the consumer is willing to bear the complexity of understanding the source structures. It can also support batch-oriented movement of larger data volumes without requiring the source system to expose new service endpoints.

Considerations

This methodology requires more than agreement in principle. It requires the deliberate design of a source-to-target extraction channel for each environment. In practice this usually means dedicated database views or procedures, associated to a dedicated database user, which is in turn associated to a dedicated service identity for that specific source-target flow. Non-production and production channels should remain separate. The channel must also account for extraction timing, watermarking, deletion handling, schema change management, and operational impact on the source environment. Credentials, certificates, or other service identity material must have defined issuance, rotation, expiry, and recovery processes. Monitoring, audit, and access review must be sufficient for someone to show why the path remains secure, why it is still least-privilege, and how it will continue to function when credentials, certificates, schemas, or infrastructure change. Where views are introduced to support ETL, they are often used mainly as a security wrapper around internal tables rather than as a deliberate conceptual projection of internal logical entities. That distinction matters. If the view merely exposes the internal model in a slightly safer way, then the consumer is still dependent on internal structures without any real brokering for maintainability or evolvability. Where files or cloud storage are used as transient staging surfaces within the ETL process, broad standing access should be avoided. Short-lived delegated access can be a viable alternative, but only if the mechanism for issuing and retrieving that access is itself secured, auditable, and operationally supportable.

Disadvantages

Direct ETL tightly couples the consumer to the source system's storage design, timing characteristics, and operational behaviour. Even where views are used, those views often remain projections of the internal logical model only lightly wrapped for access control rather than brokered external contracts. The result is an external dependency on the internal model with little room for maintainable evolution. That makes source change harder, increases downstream reconciliation when different consumers extract and reshape the same data independently, and reinforces the legacy services and organisational silos that grow around them. It should therefore be treated as a constrained or transitional methodology, not as a mature interoperability pattern.

Integration Agents

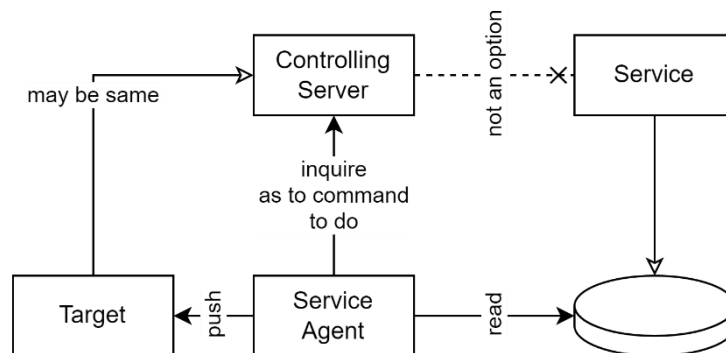


Figure 11: ETL Service Agent Approach

An integration agent installs executable capability close to the source system so that data or operations can be reached without the source exposing a proper external interface. In practice this is often used to query a datastore, execute an extraction process, or bridge a protocol gap from within the source environment itself.

A common example involves datastores—most often relational databases—which can be queried if a service agent is installed on a device within the same network as the target system.

Service agents may function in two primary ways: they can perform tasks independently, following instructions that were set during deployment; or they can operate under real-time remote control, receiving commands from a central controlling server.

This approach enables direct interaction with system internals, providing an alternative pathway for data access or process automation when standard integration methods are not feasible.

Note:

A remotely controlled service agent effectively changes the process from a PUSH to a PULL approach, similar to what an integration API approach provides.

In both cases, communication between server and target destination, and/or communication between service agent and controlling server, if any, are required to be over secure channels.

Additionally, the service agent must identify itself using credentials. These credentials are required to be initially provided to the client in a secure manner, then persisted by it in a secure manner (e.g.: using an installed or default system encrypted keystore)³.

³ Use of encrypted cleartext is not appropriate for sensitive data.

Advantages

This method can be effective where direct integration APIs are unavailable and where the source environment can support a controlled agent. Its value lies in making some otherwise inaccessible integrations technically possible, not in improving the underlying architecture.

Azure provides ready made solutions for these types of tasks: both Azure Data Factory and Data Fabric components.

Considerations

Using an integration agent requires agreement to place executable capability inside the source environment, which is not a small concession. The agent still needs source-specific identities, controlled permissions, secure storage of credentials, monitoring, audit, and lifecycle management. It may also depend on custom views, procedures, or extraction logic in the source system, which means the consumer has still not escaped source-system dependency. An agent may make the path workable, but it does not make the boundary mature.

Disadvantage

Integration agents add another operational estate to design, deploy, secure, and preserve. They usually require specific infrastructure, source-specific identities, custom extraction logic, and continuing coordination with the source environment. The result is often a technically workable path that remains tightly coupled, operationally heavy, and difficult to evolve. It should therefore be treated as a constrained workaround, not as a mature interoperability pattern.

Integration Agents through Tunnel Agents

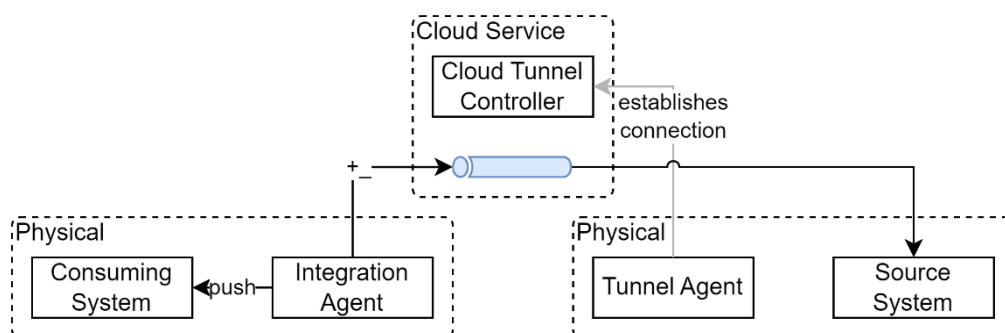


Figure 12: Integration via a Tunnel Agent

Where even a normal integration agent cannot be installed or supported, some estates resort to tunnel agents that create a controlled path from the consumer into the source environment through a third-party mediated tunnel. This makes the inaccessible

reachable, but at the cost of introducing yet another dependency boundary that must be trusted and governed.

If permission can be secured, a virtual tunnel agent (for example, Cloudflare's "cloudflared" agent, which is compatible with both Windows and Linux environments) is installed on a device within the remote organisation's network, in proximity to the source database. This agent initiates an outbound connection to its cloud-based controller (such as Cloudflare), establishing a secure virtual tunnel.

In the consumer's network, an integration agent is installed. Database queries are then routed through Cloudflare and the remote tunnel agent, ultimately reaching the target data source.

Advantages

Tunnel agents can make some integrations possible where platform constraints or organisational policy prevent other agent patterns. They may reduce the need for direct inbound network openings, but that should be understood as an enabling compromise, not a strategic strength.

Minimal Firewall Reconfiguration: Because tunnel agents typically initiate outbound connections, there is often no need to open inbound firewall ports, reducing security risk and simplifying setup.

Rapid Deployment: The installation and configuration process for tunnel agents is generally straightforward and can be completed quickly, enabling faster project rollouts.

Granular Access Controls: Many tunnel agent solutions offer detailed access control features, allowing organizations to restrict agent operations to specific resources or times.

Reduced Need for VPNs: Secure cloud based tunnels can eliminate the necessity for complex VPN infrastructure, streamlining network architecture and reducing maintenance overhead.

Considerations

Tunnel-agent patterns concentrate trust in several places at once: the remote organisation must allow an agent into its environment, the third-party tunnel provider must be trusted to operate securely and reliably, and the consumer must still manage source-specific identities, permissions, monitoring, and change. Network convenience does not remove the need for strong governance over who can reach what, by which route, and under whose operational responsibility.

Disadvantages

Tunnel agents add complexity, third-party dependency, and trust concentration to an already weak boundary. Performance can still suffer, troubleshooting becomes harder, and the organisation is now dependent not only on the source environment but also on the tunnel service and its operating assumptions. This may be necessary in constrained cases, but it is still a last-resort pattern rather than an example of mature design.

Anti-patterns to Avoid

Constraints sometimes force the hand, especially where older systems have no suitable interfaces and cannot realistically be changed in the near term. Even so, some patterns should be recognised as anti-patterns because of the impacts they create. A common anti-pattern is to choose an approach by what happens to be available rather than by what the interface is for, or by what topology the estate should mature toward. This often leads teams to reuse UI APIs for system integration, to expose internal database structures directly, or to let available integration suites dictate the architecture.

1. Proliferating point-to-point arrangements as though they were strategic architecture.
2. Exposing internal logical models as if they were external conceptual contracts.
3. Using database views as a substitute for proper interface design.
4. Coupling consumers directly to storage schema or operational behaviour.
5. Naming the integration by the source system or technology as though the system were the point. Integration is not about the source. It is about the purpose of the exchange and the type of information needed.
6. Confusing API categories by treating UI-facing, interoperability, and reporting interfaces as though they were interchangeable.
7. Allowing vendor products, available tooling, or temporary funding constraints to become the reason a weak pattern is perpetuated.
8. Recognising the need for a governed operational boundary, yet continuing to extend a vendor-managed or managed-point-to-point pattern simply because it is already running.
9. Where constrained techniques are unavoidable, introducing them without explicit risk acceptance, compensating controls, and a clear documented understanding that they are compromises rather than exemplars.

Recommendations for Maturing Integration and Interoperability Capability

1. Prefer brokered and governed information access over continued growth in point-to-point dependencies. Point-to-point integration embeds the importance of the source system into downstream design instead of focusing consumers on the value of the information itself irrespective of source.
2. Where interoperability must persist across time and multiple consumers, prefer operational data hubs over preserving or disguising point-to-point dependency. Their value is not only control. It is the ability to persist, merge once, and share more valuable downstream information objects across multiple consumers.

3. Prefer smaller, domain-aligned, distributed operational data hubs over one universal hub. This reduces risk, better serves local needs, and makes organisational change, restructuring, or divestment materially easier to absorb.
4. Encourage purpose-designed integration interfaces wherever they can be funded and built. Where direct source-system interfaces cannot realistically be created, the estate should still mature toward brokered boundaries rather than normalising point-to-point extraction or vendor-managed workaround boundaries.
5. Do not allow constrained and storage-led exchange methodologies to become normal architecture through drift. Governance should only approve such approaches where they are explicitly recorded as technical debt in both the project and organisational technical-debt registers, and only where a credible plan exists to pay that debt down. Technical debt that is never paid is not debt at all. It is theft from the future.
6. Treat the design and documentation effort required to justify weak patterns as a secondary signal. If a more endorsable approach is genuinely available, use it. If the only way to justify a constrained pattern is through extensive explanatory effort, that is maybe evidence that the architecture is already trying to explain away a better option.
7. Do not preserve source-system primacy for its own sake. The objective is not to secure funding merely to modernise interfaces, nor to keep one system treated as special forever. The objective is to improve the usefulness, governability, and decision value of information across the estate.

Conclusion

Migration to maturity is a process, but the direction still matters. The objective is not to preserve source-system primacy, to defend inherited boundaries, or to secure funding merely to modernise interfaces for their own sake. The objective is better decisions made from information.

For that reason, design should not become trapped in polemics about record-of-truth purity when the more important question is how information is brokered, combined, governed, and made useful in practice. Systems of record matter, but they are not the end of the story. Information exists to support action, judgement, and service, not to preserve the prestige of the system that first captured it.

The mature direction is therefore away from direct dependency on source systems and toward governed, brokered, and distributable information use. In many settings, record of use is more valuable than argument about record of origin. Architecture that forgets this will continue to spend money preserving yesterday's boundaries instead of improving tomorrow's decisions.

Appendices

Appendix A - Document Information

Authors & Contributors

Author: Sky Sigal, Solution Architect

Versions

- 0.1 Initial draft.
- 0.2 Glossary diagrams added.
- 1.0 Updated for Te Rito
- 1.1 Updated for Pourato
- 1.2 Updated Methodologies
- 1.3 Updated Recommendations and Anti-Patterns

Images

Figure 1: Point to Point Topologies	9
Figure 2: 2 Phase addition of Gateway, then replacement by Gateway	11
Figure 3: Integration Platform managed Topologies	12
Figure 4: Data Warehouse as Source	13
Figure 5: Data Store based Topologies	14
Figure 6: Operational Data Hubs	15
Figure 7: Distributed Operational Data Hubs	16
Figure 8: Web hooks based integration	21
Figure 9: API integration approach	22
Figure 10: UI API Reuse Approach	25
Figure 11: ETL Service Agent Approach	31
Figure 12: Integration via a Tunnel Agent	32

Tables

No table of figures entries found.

References

This document draws on the following authoritative references for interface design, API description, and API security: ISO/IEC 20802-1:2016 Information technology — Open data protocol (OData) v4.0 — Part 1: Core; OASIS OData v4.0; OASIS OData Version 4.01; OpenAPI Initiative; OpenAPI Specification; RFC 6749 The OAuth 2.0 Authorization Framework; RFC 9700 Best Current Practice for OAuth 2.0 Security; and RFC 7519 JSON Web Token (JWT).

Review Distribution

The document was distributed for review as below:

Identity	Notes
Sandy Britain, Enterprise Architect	
Russell Campbell, Project Manager	
Amy Orr, Data Domain Architect	
Duncan Watson, Enterprise Architect	
Carl Klitscher, Senior Solution Architect	
Vincent Wierdsma, Lead Developer	
Peter Martin, Lead Developer	

Audience

The document is technical in nature, but parts are expected to be read and/or validated by a non-technical audience.

Structure

Where possible, the document structure is guided by either ISO-* standards or best practice.

Diagrams

Diagrams are developed for a wide audience. Unless specifically for a technical audience, where the use of industry standard diagram types (ArchiMate, UML, C4), is appropriate, diagrams are developed as simple “box & line” monochrome diagrams.

Terms

Refer to the project's Glossary.

Asynchronous JavaScript and XML (AJAX) : a front-end development technology. A key technology for developing [SPA](#) based service clients.

Application Programming Interface (API) : a system's interface for remote access by another person. Preferably it authenticates authorises, audits, validates access.

Create Retrieve Update Delete (CRUD) : an acronym for the primary operations on data stores.

Character Separated Values (CSV) : a text-based information exchange format.

Extract, Transform, Load (ETL) : an acronym for a common process of transferring information between systems, where one or both parties do not have an integration API.

Graphical User Interface (GUI) : a form of a system's [User Interface \(UI\)](#). Traditionally [WIMP](#).

GraphQL : a often used but non-standards based open source approach to qualifying some [REST](#) operations. See [OData](#).

IT : acronym for Information, using Technology to automate and facilitate its management. IT is a subset of ICT.

ICT : acronym for Information & Communication Technology, the domain of defining Information elements and using technology to automate their communication between entities.

JavaScript Object Notation (JSON) : a serialisation format for describing objects used for communicating between systems.

Long polling⁴ : Long polling is used in real-time web applications to achieve near-instantaneous communication between clients and the web services. It is particularly useful in chat and messaging applications where real-time updates are crucial. When used by service agents behind firewalls it has the advantage that it is permitted to call out to the external command server, whereas the command server would be blocked calling into an agent.

Minimum Viable Product (MVP) : A deliverable that meets the minimum set of functional and non-functional requirements expected by stakeholders.

Missing Valuable Planning (MVP) : the state of Agile projects to which no project planning has been applied.

ODATA : an international standards-based method of qualifying [REST](#) operations (e.g., filtering, sorting, projecting, paging). Contrast with [GraphQL](#).

⁴ [What is Long Polling? | PubNub](#)

OpenAPI : a standard for describing [REST](#) based interfaces in a machine-readable format.

PULL : an integration approach where the service client requests what it wishes for from a service.

PUSH : an integration approach where the service pushed to service clients without confirmation that the server desires it.

Remote Procedure Call (RPC) : an API architecture pattern for [APIs](#) that permits the invocation of Procedures/Functions. Contrast with [REST](#), an [API](#) architectural pattern that provides methods to manipulate Resources instead.

REsource State Transfer (REST) : a messaging protocol used to communicate between systems.

Simple Object Access Protocol (SOAP) : a messaging protocol used to communicate between systems. See [REST](#).

Singe Page App (SPA) : a modern approach to web solutions where the whole site, minus data is transmitted to the browser, calling back for data as needed to be displayed. SPAs are noted for improving responsiveness while decreasing server resource utilisation as well as IO bandwidth, therefore cost.

Textual User Interface (TUI) : a console form of [UI](#).

User Interface (UI) : an interface for use by system users (as opposed to a remote system). May be a [TUI](#) or [GUI](#).

Web Application Firewall (WAF) : a security firewall specialised with analysing web-based HTTP/S traffic.

Windows, Icons, Mouse, Pointer (WIMP) : an acronym for a type of [GUI](#).

Appendix B - Information Flow Modelled as Reticulated and Storm Water

A useful way to explain the difference between operational interoperability, copied stores, and analytical accumulation is to think about information flow in the same way as a city's water infrastructure.

Reticulated water is governed supply: intentionally directed, controlled, and fit for direct use. Stormwater is runoff: water that has already flowed away from its operational point of use and has been collected downstream for observation, drainage, or later treatment.

The analogy is not exact, but it helps distinguish information that is intentionally brokered for operational use from information that has merely accumulated elsewhere.

In that analogy, a data warehouse is closer to stormwater runoff collected downstream. It may be valuable for analysis, trending, and reporting, but it is not the same thing as a governed operational supply.

An operational data hub is more like an underground cistern with controlled access: information is intentionally gathered, brokered, and made available through a managed boundary for approved use.

A datastore used directly for integration is more like an open-topped cistern. The water is there, but access is less governed, there is no deliberate logic layer shaping how it should be accessed or consumed, and downstream consumers are exposed too directly to what has merely been stored.

The point of the analogy is not to romanticise infrastructure. It is to show that not every place where information can be found is a suitable place from which to serve it operationally. Mature interoperability depends on governed supply for use, not merely on accessibility of stored or copied data.

The same analogy can also help explain other topology choices.

Point-to-point integration is like neighbours running hoses across a cul-de-sac to help one another keep water flowing. It can work for isolated needs, but as more hoses are added the arrangement becomes tangled, fragile, and hard to govern.

A gateway-mediated topology is like requiring all of those hoses to pass through one neighbour's property before redistribution. This improves visibility and control, but it also adds another checkpoint, another point of delay, and another dependency without necessarily improving the quality of the underlying supply.

An integration platform is closer to a specialised business that rents, routes, and manages the hoses for everyone. That may reduce repeated effort, but it does not itself improve the water system. It makes income from the neighbours' disorganisation, puts distance between them and the underlying problem, and can hide weak architecture under an ongoing bill rather than maturing it, diverting precious attention, analysis, funding and effort away from fixing the underlying cause.

Appendix C - Extreme Case of Boundary, Schema, and Control Muddling

An extreme form of architectural muddling occurs when an organisation funds or legitimises a sector-facing or partner-facing interoperability boundary, but does not retain practical control over the shared schema, rights to use the data, or the long-term shape of the platform through which that boundary operates. The result can look coordinated on the surface while remaining strategically weak underneath.

The pattern becomes worse when the need for a governed operational boundary is already understood, yet the corrective design is deferred while the weaker boundary

continues to expand. Additional integrations are then pulled into the same arrangement because it is already running, politically safer, or easier to extend than to replace. The architecture thereby becomes more entrenched precisely after its weakness is known.

In such a case, the organisation may carry the cost of the dependency without receiving the strategic benefit that should have justified it. It may not receive the data in a controlled internal shape, may not hold practical rights to use the data as intended, and may not be able to move the boundary later without material operational or reputational difficulty. Hosted delivery does not remove that risk. It merely changes where the platform is run.

The architectural lesson is simple. A vendor can operate a boundary. That does not mean the organisation or sector should surrender control of the boundary itself. If schema control, data rights, and exit are not retained, then coordination has not been achieved. It has only been rented.